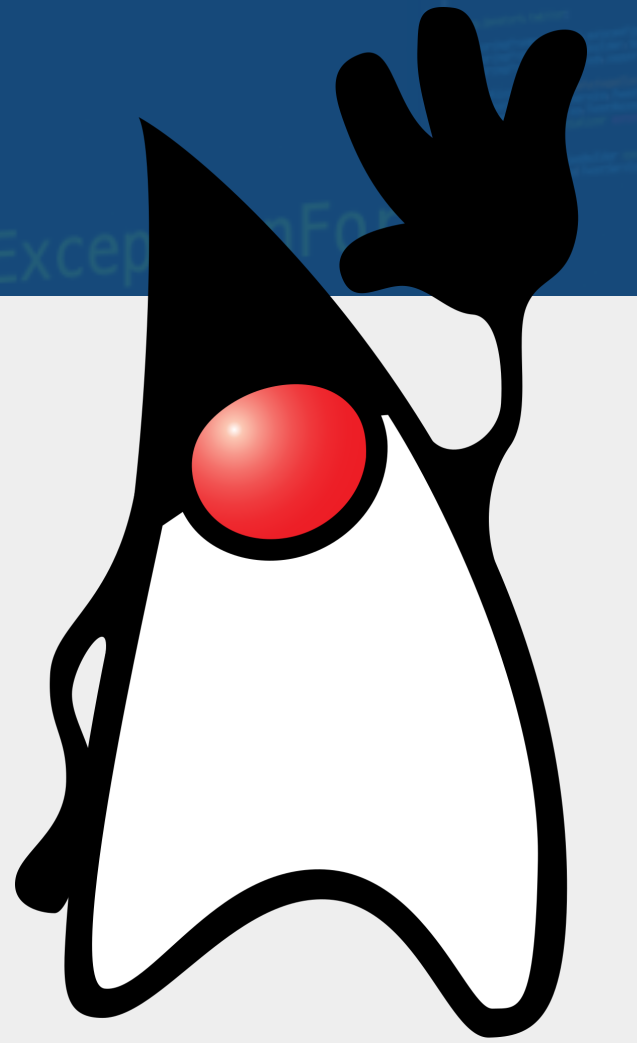


Java Programlama Dili

4. Bölüm

Kaynak Kod Yapısı

Akın Kaldıroğlu





- Bu bölümde daha rahat ilerleyebilmek için Java'nın kaynak kod yapısını (source code structure) ele alacağız
- Böylece daha detaylı konuları işlerken verilecek kod örnekleri çok daha anlaşılır olacaktır
- Burada ele alınan pek çok nokta ilerdeki bölümlerde detaylandırılacaktır

Konular



- **Kodun Bileşenleri**
 - Tanıtma
 - İfade
 - Cümle
 - Blok
- **Sınıf Bloğu**
 - Nesne Yaratma
- **Kaynak Kod Dosyası**
 - Derleme ve Çalıştırma
- **Kısa Kaynak Dosyaları ve Nesne Main Metotları**
 - Kısa Kaynak Dosyaları
- **main Metot**
 - Nesne main Metotları
- **Ödevler**
- **Bundan Sonra?**

Kodun Bileşenleri



Kodun Bileşenleri



- Kodun bileşenleri dendiğinde akla gelen ana yapılar şunlardır:
 - Tanıtma (Declaration)
 - İfade (Expression)
 - Cümle (Statement)
 - Blok (Block)



Tanitma

Tanıtma - I



- Tanıtma (tanıtım, declaration), belirteciyle (isim, identifier) bir program yapısını sunmak, tanıtmak demektir
- Burada sunulan, tanıtılan taraf derleyicidir `int count // Variable declaration`
- Tanıtmada kullanılan belirteç ya da isim, sunulan yapıyı temsil etmek veya onu göstermek üzere sonrasında programda kullanılır
- Tanıtmada kullanılan belirtecın tabi olarak geçerli olması, örneğin bir anahtar kelime (keyword) olmaması gerekir

Tanıtma - II



- Farklı tanıtma şekillerine örnekler:

```
int count // Variable declaration, later it is assigned and read
```

```
package org.javaturk.jp.ch04.car // Package declaration later it is imported
```

```
import org.javaturk.jp.ch04.car.Car // Import declaration, later Car is used
```

```
class Car{ // Class declaration, later it is instantiated  
    ...  
}
```

```
void accelerate(int newSpeed){ } // Method declaration, later it is invoked
```

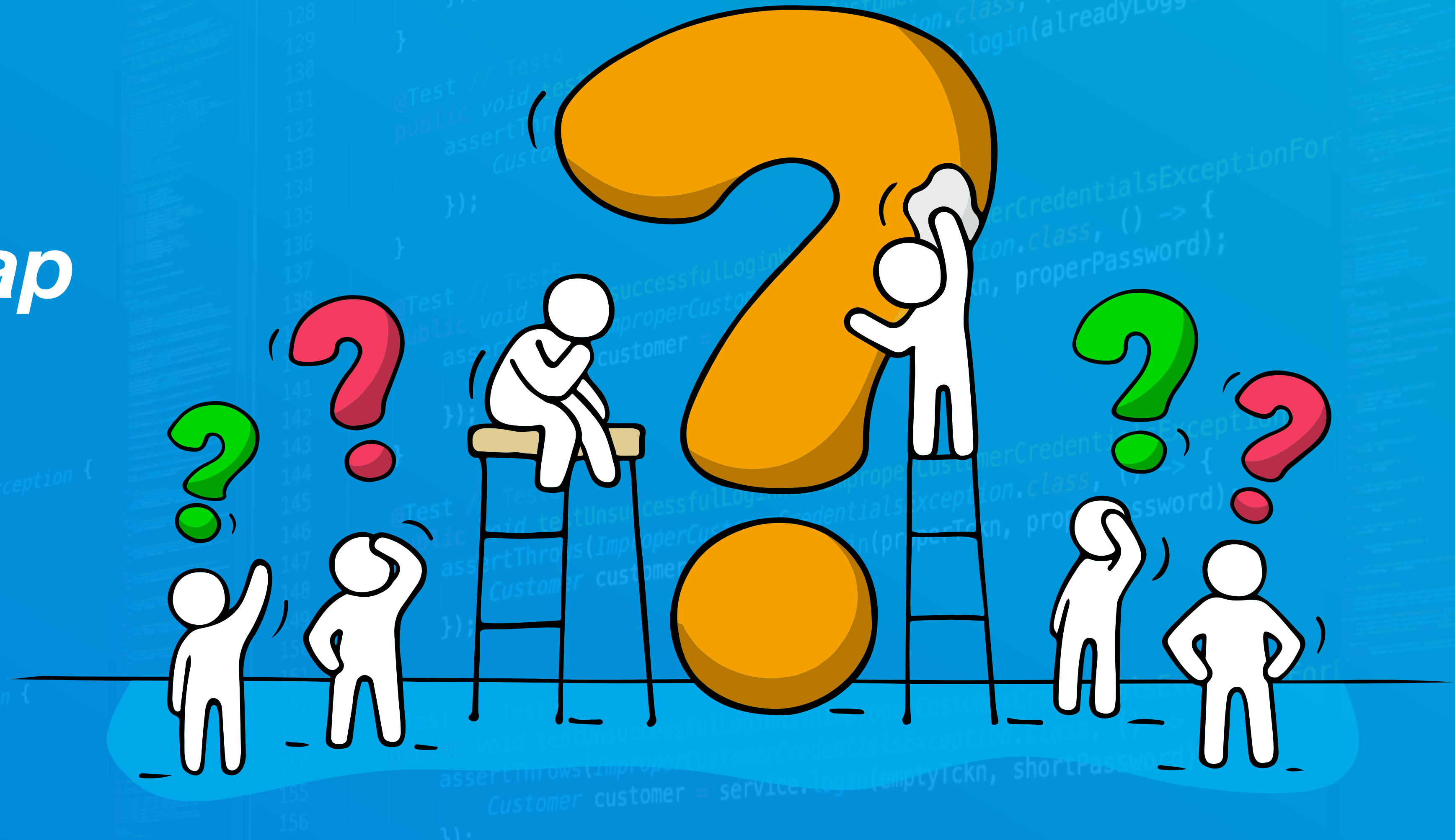


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak tanıtmalara örnekler bulun

Soru ve Cevap Zamanı!





ifade

İfade (Expression) - I



- Bir değer üretecek şekilde, değişkenler (variables), işlemciler (operators), metot çağrıları vb. öğelerden oluşan yapıya **ifade (expression)** denir

- Üretilen sonuç bazen bir değerdir yani ifade bir değer üretir

```
sayac++ // Simple expression, increasing sayac by 1  
// There are two expressions in this, sayac's values before and after ++
```

- bazen bir değişkendir (C'deki *lvalue*), ifade bir değer üretip bir değişkene atar

```
sayac = 2 // Simple expression, its value is the value of sayac, which is 2  
// There are two expressions in this, 2 and sayac
```

İfade (Expression) - II



- Dolayısıyla ifadeler, değer kullanılan her yerde kullanılabilir

```
// break is not an expression  
int x = break  
// Method call is an expression  
int x = square(3)
```

- Java'da ifadelerin tipleri olur ve bu tip derleme zamanında belirlenebilir
 - Değerlerin tipleri olduğu gibi
- Programların çalışması demek devamlı olarak ifadelerin değerlerinin belirlenmesi demektir

İfade (Expression) - III



- İfade türleri olarak, atama (assignment), şart (conditional), switch, metot çağrısı, lambda ifadeleri vb. sayılabilir

```
sayac = 5    // Assignment expression  
sayac++      // Post-increment expression
```

```
// conditional expression  
if(distance ≤ 1.0)
```

```
// switch expression  
int i = switch(value){  
    ...  
}
```

```
// method invocation expression  
Math.sqrt(2)
```

İfade (Expression) - IV



- Sadece tek bir durumda, ifadenin bir değeri yoktur yani ifade değer üretmez
- Bu ancak `void` döndüren bir metodun çağrılmasıyla olur
- İleride göreceğimiz gibi `void` döndüren metotlar çağrıldığı ortama bir değer döndürmez

```
// returns void
void greet(String name){
    System.out.println("Selam " + name);
}
```

```
// method invocation exp.
object.greet("Ali")
```

İfade (Expression) - V



- Birden fazla ifade bir araya gelerek **bileşik ifade (compound expression)** oluşturur
- Bir ifadede üretilen sonucun tipi, o ifadede kullanılan değişkenlerin tipine, işlemcilere ve metotlara bağlıdır

```
1 * 2 * 3           // Compound expression, 5 expressions.  
sonuc = 2 * sayac   // Compound expression, 4 expressions.  
b = (x > y) & a.f(5) // Compound expression, 7 expressions.
```

İfadelerin Değeri - I



- Java'da ifadelerin değeri tabii olarak ya derleme zamanında (compile-time) ya da çalışma zamanında (run-time) belirlenir
- Bir ifade, 3 ya da 8.5 gibi sayısal, `true` veya `false` gibi mantıksal ya da "Java" gibi `String` sabitelerinden oluşuyorsa bu durumda Java derleyicisi ifadenin değerini derleme zamanında belirlemeyi tercih eder
- Bir ifadenin değeri derleme zamanında belirlenebiliyorsa ona **sabit ifade (constant expression)** denir

```
String word = "Java" + " is " + "fun"; // Constant expression
int i = 2 * 5 + 3; // Constant expression
boolean b = 5 > 2; // Constant expression
```

İfadelerin Değeri - II



- Değerin bir metot çağrısı ile belirlenmesi ya da kullanıcının girmesi veya bir dosyadan okunması gibi, belirsizlik oluşturan bir durum varsa bu durumda ifadenin değeri ancak çalışma zamanında belirlenebilir

```
int i = Math.pow(2, 10); // Not a constant expression
int j = readFromFile(); // Not a constant expression
int k = i * j; // Not a constant expression
```

- Derleyici, çok basit bile olsa bir metot çağrısı ile oluşacak bir değeri belirlemez; bunun sebeplerinden birini ileride öğreneceğiz

```
int i = getMeFive(); // Not a constant expression
// A methods that returns a constant
int getMeFive(){ return 5; }
```

İfadelerin Değeri - III



- İki sebepten dolayı ifadelerin değerinin belirlenme zamanıyla ilgileniyoruz:
- `switch-case`'de `case`'den sonra gelen değerlerin birbirlerinden farklı olması zorunludur dolayısıyla bu değerler sabit ifade olmalıdır
- Derleyiciler çalışma zamanı performansını arttırmak amacıyla derleme zamanında mümkün olan optimizasyonları yapmak isterler

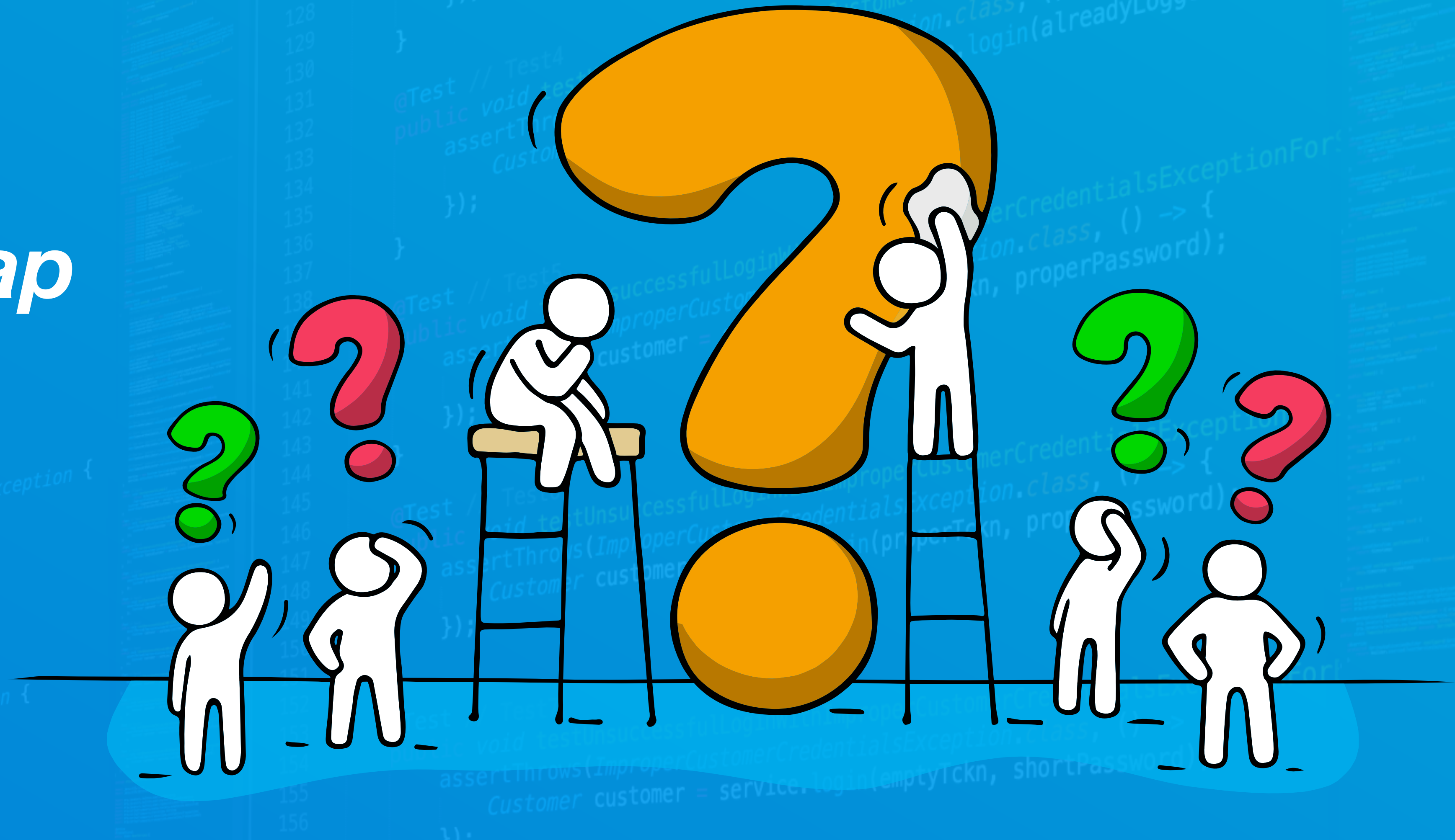
```
// Constant folding and constant propagation  
int time = 60 * 60; // Compiler does the math: int time = 3600;  
String word = "Java" + " is" + "fun"; Compiler sums it: String word = "Java is fun";
```

ValuesOfExpression



- `ValuesOfExpressions`

Soru ve Cevap Zamanı!





Cümle

Cümle (Statement) - I



- **Cümle (statement)**, programlama dili vasıtasıyla bilgisayara verilen bir emirdir
- Emirler aslında derleyici ve yorumlayıcı vasıtasıyla verilir ve her emir bir iş içindir dolayısıyla programlar emirler ile bilgisayara iş yaptırır
- Java'da cümleler genelde **;** ayıracıyla (separator) bitirilir ve bu genelde Java'nın eleştirilen taraflarından birisidir

```
int i = 5;  
i++;  
System.out.println("i");
```

```
if(a > b)  
    System.out.println("*");
```

Cümle (Statement) - II



- Cümle, dillerde en küçük **çalışma birimidir (unit of execution)**
- Yani çalışma zamanı, cümleleri art arda çalıştırarak ilerler
- Art ardalık ile programlamanın ilk zamanlarında yukarıdan aşağıya çalıştırma kastedilirken, fonksiyon, subroutine, nesne gibi soyutlamaların yarattığı dallanmalar ile daha karmaşık mekanizmalar ortaya çıkmıştır

```
Greeting greeting = new Greeting();  
String answer = greeting.greet("Hi");  
System.out.println(answer);
```



- Ama her halükarda çalışma, cümleler üzerinden ilerler

Cümle (Statement) - III



- En basit cümle, bir tanıtmaya ; eklemek ile oluşur: **tanıtma cümlesi (declaration statement)**

```
int i; // Variable declaration statement  
package org.javaturk.jp.ch04.car; // Package declaration statement
```

- Bazen, bir ya da daha fazla ifade bir araya gelip ; ile biterek de bir cümle oluşur: **ifade cümlesi (expression statement)**

```
int i = 5; // Two expressions  
int j = 2 * i++; // Five expressions  
System.out.println(j); // An expression
```

```
// if is a statement not an expression  
if(a > b) // Three expressions  
    System.out.println("*"); // An expression
```

Cümle (Statement) - IV



- Bu anlamda cümle, ifadenin çalıştırılabilir hali olarak düşünülebilir

```
// Expression statements  
sayac = 5; // Assignment  
sayac++;   // Post-increment  
++sayac;   // Pre-increment  
sayac--;   // Post-decrement  
--sayac;   // Pre-decrement  
new Car(); // Class instance creation  
car.stop(); // Method invocation
```

Cümle (Statement) - V



- İfadelerin değeri vardır ama cümlelerin değeri yoktur, yan etkileri vardır; çalıştıklarında bir iş yaparlar
- Atama, değer arttırma, metot çağrısı yapma, karar verme, tekrarlama, vs.

```
sayac = 5; // sayac is now 5  
sayac++; // sayac is now 6
```

```
if(a > b)  
    System.out.println("*");  
// when a is greater than b the * is printed
```

```
Greeting greeting = new Greeting();  
String answer = greeting.greet("Hi");  
System.out.println(answer);
```

Cümle (Statement) - VI



- İfadeler ifadeleri içerebileceği gibi (bileşik ifadeler) cümleler cümleleri (sub-statement) hatta blokları içerebilir

```
public void f(){  
    new Blocks(){  
        public void sop() {  
            System.out.println(1);  
        }  
    };  
}
```

Cümle Tipleri - I



- Farklı türde cümleler vardır:
- **Tanıtma cümleleri (Declaration statements)**

```
int sayac;  
boolean b;  
A a;
```

- **Tanımlama cümleleri (Definition statements)**

```
int sayac = 4;  
boolean b = true;  
A a = new A();
```

Cümle Tipleri - II



- Akış kontrolü cümleleri (flow control statements)

- **if** cümlesi

```
if(x > y)
    System.out.println("x büyük");
```

- **for** cümlesi

```
for(int i = 1; i ≤ 100; i++)
    sum += i;
```

- **switch** cümlesi

```
switch (k) {
    case 1: System.out.print("one");
    case 2: System.out.print("too");
    case 3: System.out.println("many");
}
```

Cümle Tipleri - III



- Boş cümle (empty statement) 
- Bunlar dışında `for`, `assert`, `try` cümleleri gibi daha pek çok farklı türde cümle vardır
- Bunlar ileride ele alınacaklardır

Cümle ve Satır



- Her cümlenin ayrı bir satırda (line) olması zorunlu olmamakla birlikte okunabilirlik açısından iyi bir uygulamadır
- Fakat,
 - bir satırda birden fazla cümle olabilir
 - bir cümle, birden fazla satırı kaplayabilir
- Bu iki yöntemin gelenekte yeri yoktur, temiz koda terstir, kötü koku (code smell) oluşturur

```
int i = 5;  
i++;  
System.out.println(i);
```

```
int i = 5; i++; System.out.println(i);
```

```
int  
  i  
    =  
      5  
        ;
```

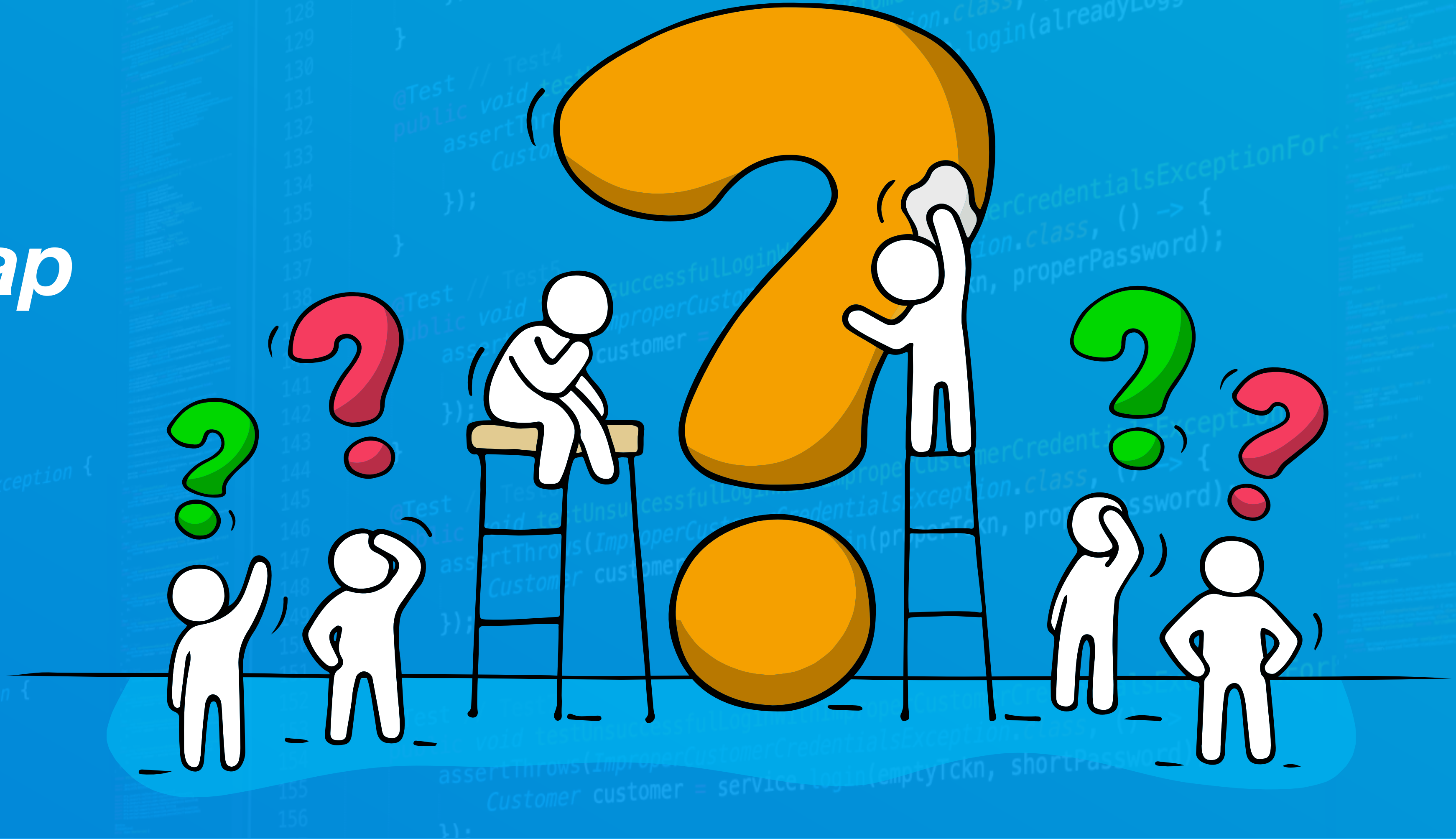
```
System  
.  
out  
.  
println(i)  
;
```

Statements



- `Statements`

Soru ve Cevap Zamanı!





Blok

Blok - I



- Sıfır ya da daha fazla sayıdaki cümlelerin, bir parantez çifti, { ve } arasında oluşturduğu yapıya **blok (block)** denir
- Bloklar, bir cümlelerin kullanıldığı her yerde kullanılabilir,
 - Cümleler ; ile biter ama bloklar ; ile bitmez
 - Tabi bunun da birkaç küçük istisnası vardır
- Java, **blok yapılı (block-structured)** bir dildir yani kodlar daima blokların içine yazılır

Blok - II



- Bloktaki cümleler yukarıdan aşağıya doğru işletilirler
- Bloklarda alt bloklar da olabilir
- Bazen cümleler blok içerebilir

```
if(x > 0){  
    if(x > y){  
        ...  
    }  
    else{  
        ...  
    }  
}
```



```
public void f(){  
    new Blocks(){  
        public void sop() {  
            System.out.println(l);  
        }  
    };  
}
```

Blok - III



- Bazen bloklar, ilişkilendirildikleri yapılardan dolayı var olurlar:
- Sınıf, metot veya `if` blokları gibi
- Bazen de bloklar, sınıf ya da metot gibi bir yapıyla ilişkilendirilmeden, sadece bir ya da daha çok cümleyi gruplamak için kullanılır

```
public class Selam{  
    public String selamSoyle(String kime){  
        String cumle;  
        if (kime != ""){  
            cumle = "Selam " + kime + " :)";  
        }  
        else {  
            cumle = "Selam " + world + " :)";  
        }  
        return cumle;  
    }  
}
```

```
public void f(){  
    ...  
    {  
        ...  
    }  
}
```

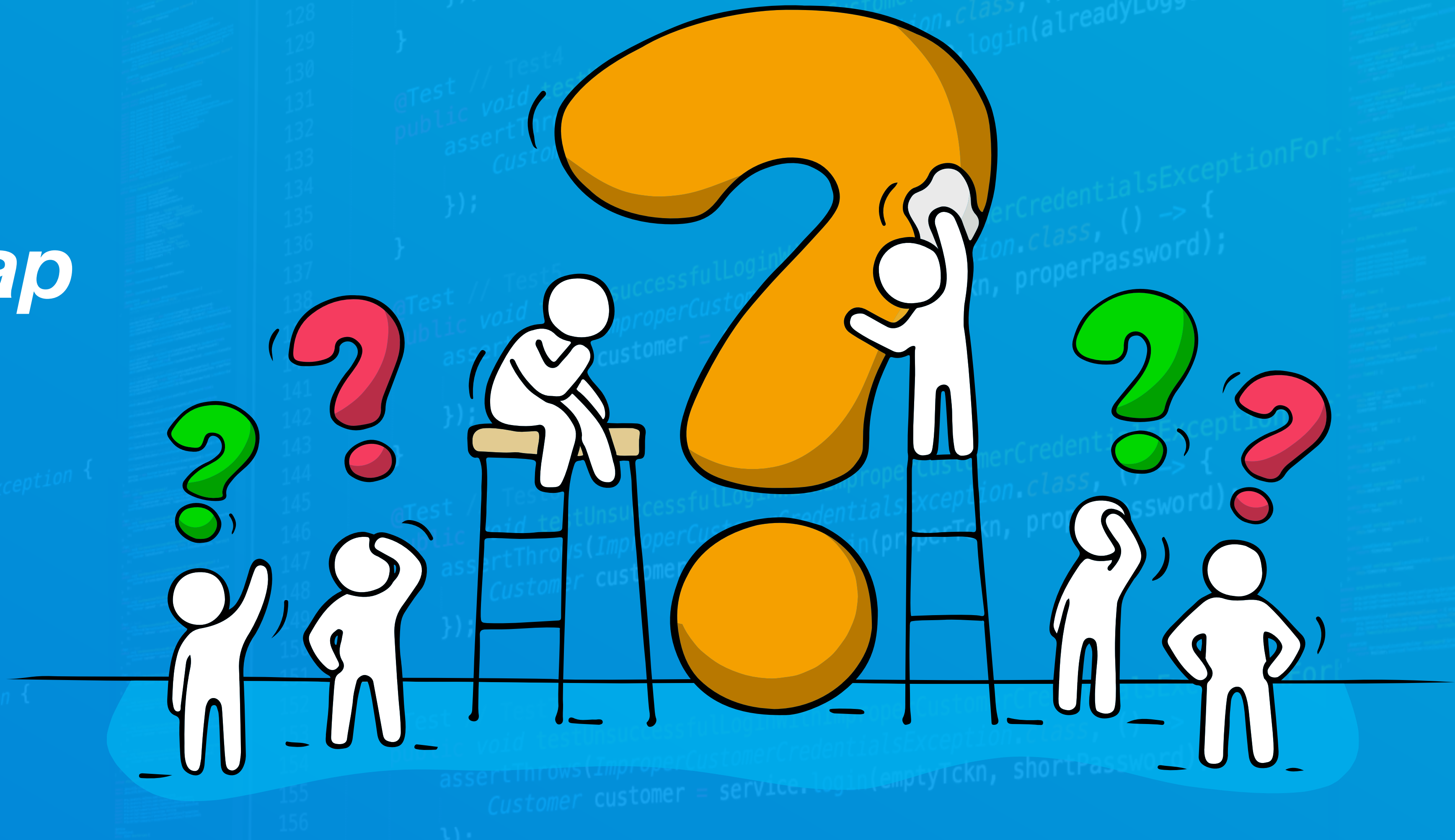


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak farklı bloklara örnekler bulun

Soru ve Cevap Zamanı!



Sınıf Bloğu



Java'da Her Şey Sınıfın İçindedir!



- Java'da en geniş blok, sınıf (class) bloğudur
- Java'da her şey sınıf içinde olmalıdır
 - Aslen bunun sadece 2 tane istisnası vardır: `import` ve `package` cümleleri
- Dolayısıyla tüm kodlar muhakkak sınıflar içinde tanımlanmalıdır
- Java 25 ile gelen Compact Source Files and Instance Main Methods ile birlikte başka istisnalar da oluştu, detaylarını ileride göreceğiz

Sınıf Tanıtma - I



- Java'da sınıf tanıtmak (class declaration) için **class** anahtar kelimesi kullanılır:

```
<niteleyici>* class <Sınıfİsmi>{  
}
```

- **Not:** *, sıfır ya da daha çok anlamındadır
- Bütün diğer bloklar daima bir sınıf bloğunda olurlar
- Metot blokları, iç sınıf (inner classes) blokları, ilk değer atama blokları (initialization blocks), vb. bütün bloklar, sınıf bloğunda bulunurlar

Sınıf Tanıtma - II



- C++'tan gelenlerin alışkanlıklarını devam ettirmelerine izin vermek için sınıf tanıtımından sonra ; kullanılabilir ama bunun gelenekte yeri yoktur

```
<niteleyici>* class <Sınıfİsmi>{  
};
```

Blocks



- **Blocks**
- Bu örnekte sınıf bloğu içerisindeki farklı blokları inceleyin

Sınıf Tanıtma - III



- Sınıfın öğeleri en temelde, **özellikler (attributes, properties)** ve **davranışlardır (behaviors)**
- Özellikler, **nesne değişkenleridir (object/instance variables)**, **alan (field)** da denir
- Davranışlar, ise **nesne metotlarıdır (object/instance methods)**

```
<niteleyici>* class <Sınıfİsmi>{  
    <değişken>*  
    <metot>*  
}
```

Sınıf Tanıtma - IV



- Zorunlu olmasa da özellikler en yukarıda yer alır, sonra metotlar gelir

```
<niteleyici>* class <Sınıfİsmi>{  
    <değişken>*  
    <metot>*  
}
```

- Sınıfın, **public** ya da **abstract** gibi bir ya da daha fazla niteleyicisi (modifier) olabilir

```
public abstract class Vehicle{  
    ...  
}
```

Niteleyici (Modifier)



- **Niteleyici (modifier)** Java'da, sınıf, metot, değişken vb. yapılara uygulanan ve uygulandığı yapının davranışını değiştiren anahtar kelimelerdir
- Farklı tipte niteleyiciler vardır, örneğin erişim niteleyicileri (access modifiers),
- Niteleyiciler, uygulandıkları yapılar ele alındıkça işleneceklerdir

Class, Enum ve Interface



- Java'da sınıf yanında, özel türde sınıf olarak düşünülebilecek ve **enum**, **record** ve **interface** anahtar kelimeleri ile tanıtılan tipler de vardır
- Diğer özellikler açısından ayrışmakla beraber, her şeyin kendi içinde ifade edildiği en geniş bloğu oluşturmak açısından, bu üç tip sınıf ile eşdeğerdir

Özellik Tanımlama - I



- Özellikler (attribute, property) değişkenlerle ifade edilir; dolayısıyla tipi, ismi ve ilk değeriyle, değişken olarak tanımlanır

```
<niteleyici>* <tip> <değişkenİsmi> = [ilk değer];
```

- **Java'da her değişkenin bir tipi (type) olmak zorundadır,**
- Değişkene ilk değer (initial value) vermek zorunlu değildir, isteğe bağlı olarak verilebilir,
- Değişkenler için de niteleyiciler vardır

Özellik Tanımlama - II



```
<niteleyici>* <tip> <değişkenİsmi> = [ilk değer];
```

```
public int age = 0;  
private boolean convertible = false;  
final static String name;
```

```
String make;  
String model;  
String year;  
int speed;  
int distance;  
boolean convertible ;
```

Metot Tanımlama - I



- Metotlar (methods), iş yapan yapılardır, nesnelerin davranışlarını (behavior) ya da sorumluluklarını (responsibility) yerine getirirler

```
<niteleyici>* <dönüş tipi> <metotİsmi>(<argüman>*){  
    <cümle>*  
}
```

- Metotlar zorunlu olmamakla birlikte, dışarıdan argüman olarak veri alırlar ve çağrıldıkları ortama veri geri döndürürler
- Metotlar için de niteleyiciler vardır

Metot Tanımlama - II



- Metodun isim ve argümanlarından oluşan bilgisine **signature (imza)** denir

```
<niteleyici>* <dönüş tipi> <metotİsmi>(<Argüman>*){  
    <ifade>*  
}
```

- İmzaya dönüş tipi de katılırsa metodun **interface**'i (**arayüzü**) elde edilir

```
<niteleyici>* <dönüş tipi> <metotİsmi>(<Argüman>*){  
    <ifade>*  
}
```

- Geriye metodun iş yaptığı kısım yani bloğu kalır; ona da **body** denir

```
<niteleyici>* <dönüş tipi> <metotİsmi>(<Argüman>*){  
    <ifade>*  
}
```

Metot Tanımlama - III



```
public String selamSoyle(String kime){  
    String cumle;  
    if (kime != "")  
        cumle = "Selam " + kime + " :)";  
    else  
        cumle = "Selam " + world + " :)";  
    return cumle;  
}
```

```
public double go(int newDistance) {  
    distance += newDistance;  
    double period = (double) newDistance / speed;  
    return period;  
}
```

```
public void accelerate(int newSpeed) {  
    speed = newSpeed;  
}
```

```
public void stop() {  
    speed = 0;  
}
```

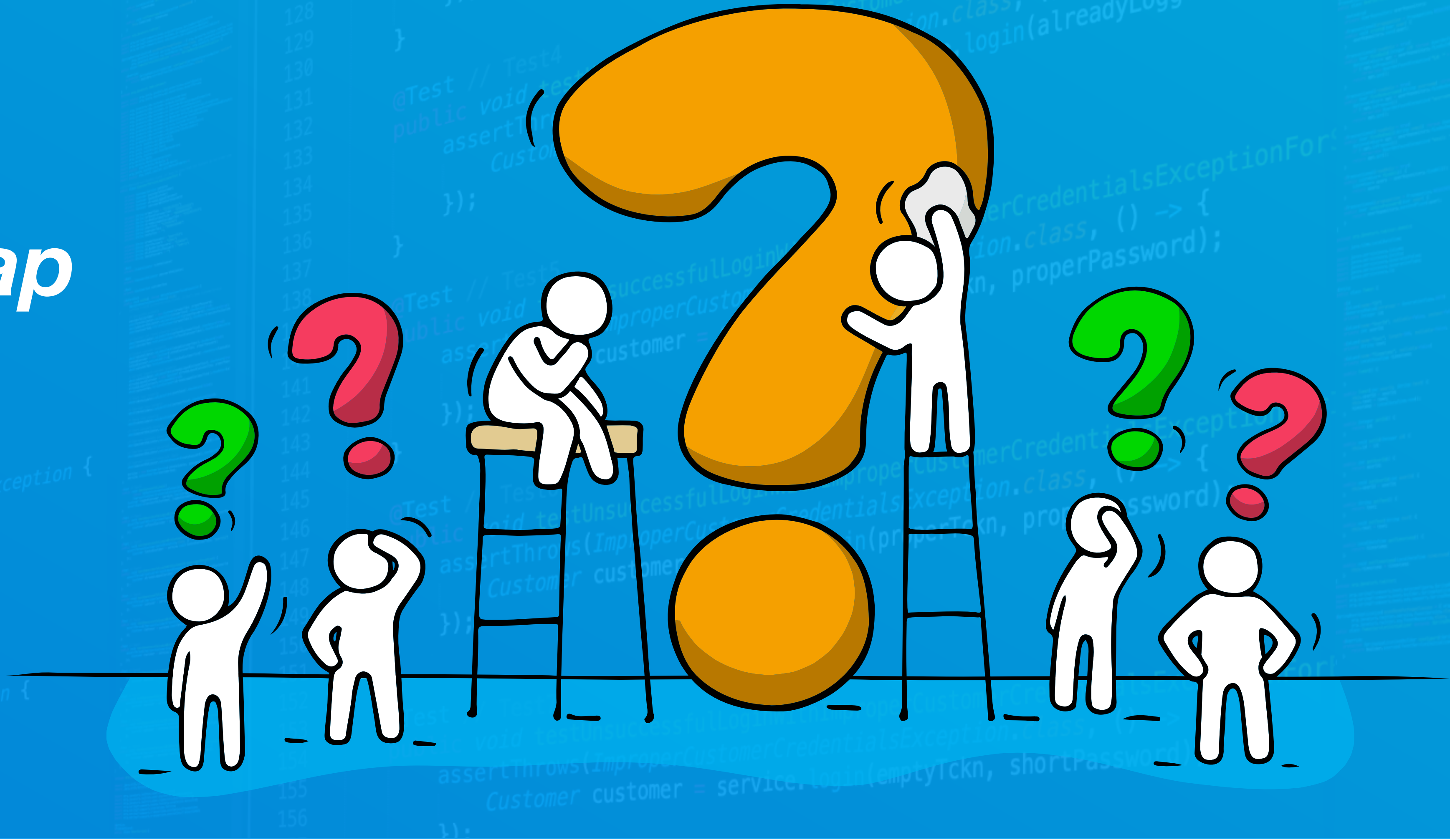


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak tanımlanan sınıf(lar)ı inceleyin

Soru ve Cevap Zamanı!





Nesne Yaratma

Nesne Yaratma (Instantiation) - I



- Nesne-merkezli dillerde en temel iş, nesne (object, instance) yaratmaktır (object creation ya da instantiation (örnekleme))
- Nesneler sınıflardan oluşturulur, sınıflar, nesneleri yaratılabilsin diye tanıtılır
- Nesne yaratmak için **new** anahtar kelimesi kullanılır `new Selam();`
- **new** sadece nesne yaratmakta kullanılır ve kendisinden sonra daima, yaratılacak nesnenin kurucu metot çağrısı (constructor call) gelir

Nesne Yaratma (Instantiation) - II



- Yaratılan nesneyi, kendisine ulaşmakta kullanılacak olan ve nesne ile aynı tipte olan bir referansa atamak gereklidir

```
Se1am reference = new Se1am();
```

- Sağ tarafta oluşturulan nesne, sol taraftaki **Se1am** tipinden **reference** isimli bir referansa atanmaktadır
- Şu ana kadar *reference* yerine *nesne* ya da *object* gibi isimler kullanıldı, bundan sonra nesne ile referansı ayırt edilecek ve daha sonra ayrıntılı işlenecektir

Nesne Yaratma (Instantiation) - II



- Nesne oluşturulunca, bellekte nesneye yer ayrılmış ve bu yerin adresi nesnenin referansına atanmış demektir
- Nesnenin sınıfında tanımlanan özellikler ve metotlara, nesne üzerinde erişilebilir

```
Selam reference = new Selam();  
reference.selamSoyle("Zeynep");
```

Car & CarTest



- `car.Car` ve `CarTest`
- Bu örnekte `Car` ve `CarTest` sınıfındaki değişkenleri ve metotları inceleyin

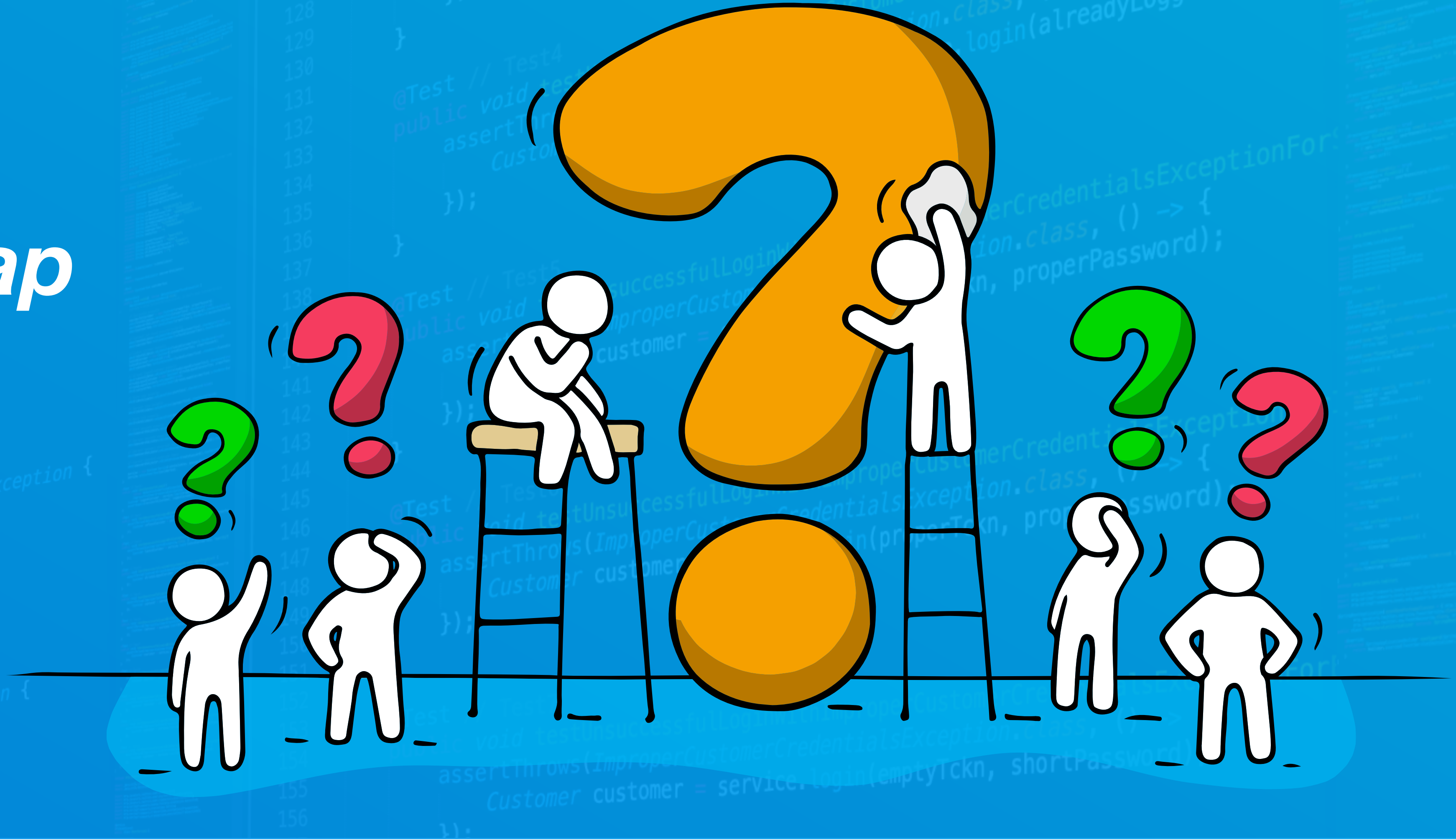


Ödev



- `NumberGuessingGame.java` dosyalarından bir tanesini kullanarak tanımlanan sınıf(lar)ı inceleyin yaratılan nesneleri ve üzerinde erişilen yapıları inceleyin

Soru ve Cevap Zamanı!



Kaynak Kod Dosyası



Kaynak Kod Dosyası



- Java'nın kaynak kodu `java` uzantılı dosyalarda tanımlanır
- Bu dosyalara **kaynak kod dosyası (source code file)** ya da **kaynak dosyası (source file)** denir
- Aksi söylenmedikçe biz de *dosya* ile Java kaynak kod dosyasını kastedeceğiz

```
Vehicle.java  
FrontController.java  
Selam.java  
SelamTest.java  
Main.java
```

Kaynak Dosya Yapısı - I



- Kaynak dosyasında sınıf(lar) tanımlanır
- Sınıfın, paket (**package**) bilgisi ile koduna dahil edeceği (**import**) diğer yapıların cümleleri, sınıf dışında ve daima sınıfın öncesinde tanımlanır
- Yani **package** ve **import** cümleleri (statement) sınıftan önce tanımlanır
- Sınıfın öğeleri olan değişkenler ve metotların sadece sınıf bloğunda tanımlanabileceğini daha önce görmüştük

```
package <paketİsmi>;  
import <tip>*<br><br><niteleyici>* class <Sınıfİsmi>{  
    <değişken>*<br>    <metot>*<br>}
```

Kaynak Dosya Yapısı - II



- Java 25 ile bunun istisnalarının oluştuğunu biliyoruz ama şimdilik bu şekilde kabul edelim

```
package <paketİsmi>;  
import <tip>*  
  
<niteleyici>* class <Sınıfİsmi>{  
    <değişken>*  
    <metot>*  
}
```

Kaynak Dosya Yapısı - II



- Kaynak dosyasında **package** ve **import** cümleleri zorunlu değildir, varsa **package** ifadesi sadece bir tane olabilir ama istenildiği kadar **import** yapılabilir
- **package** cümlesi bir kaynak kodunda varsa ilk çalışan kod olur, **package** cümlesinden önce sadece boşluk ve yorum olabilir
- Dolayısıyla varsa **import** cümlesi ya da cümleleri, varsa **package** cümlesinden sonra gelmelidir

```
package <paketİsmi>;
import <tip1>;
...
import <tipN>;

<niteleyici>* class <Sınıfİsmi>{
    <değişken>*
    <metot>*
}
```

Kaynak Dosya Yapısı - IV



- Bir Java kaynak dosyasında birden fazla sınıf tanımlanabilir
- Fakat her sınıfın kendi kaynak dosyasında yer alması yönetim açısından daha rahattır; pek çok geliştiricinin aynı anda bir sürü sınıf üzerinde çalıştığını düşünün!
- Bunu daha önce konuşmuştuk
- Dolayısıyla yandaki sınıfları ayrı ayrı **MyClass.java**, **Sınıfİsmi2.java** vb. dosyalarda tanımlamak yönetimlerini kolaylaştırır

```
<package cümlesi>

<import cümlesi>*

public class MyClass{
    ...
}

class <Sınıfİsmi2>{
    ...
}

...
class <SınıfİsmiN>{
    ...
}
```

Dosya ismi: **MyClass.java**

Kaynak Dosya Yapısı - V



- Yandaki kod `MyClass.java` isimli dosyada bulunmalıdır
- Çünkü `public` sınıf, kendi ismini taşıyan dosyada yer almak zorundadır
- Bir kaynak dosyasında istenildiği kadar `public` olmayan (*non-public*) sınıf tanımlanabilir
- İsimsiz ya da örtük sınıflar da herhangi bir dosya ismine sahip olabilir, ileride ele alınacak

```
<package cümlesi>

<import cümlesi>*

public class MyClass{
    ...
}

class <Sınıfİsmi2>{
    ...
}

...
class <SınıfİsmiN>{
    ...
}
```

Dosya ismi: `MyClass.java`

public Sınıf - I



- **public** sınıf niteleyicisi, sınıfa her yerden erişilebileceğini ifade eder
- **public** sınıf, kendi ismini taşıyan bir dosyada yer almak zorundadır
- Yani her Java kaynak kodu dosyasında en fazla bir tane **public** sınıf olabilir
- **public** niteleyicisinin burada sadece sınıflar için kullanımı ele alınmaktadır, sınıflar dışında alanlar ve metotlar için kullanımı ve diğer detayları ileride gelecektir

Soru



- Bir Java kaynak dosyasında en az kaç sınıf bulunabilir?

Cevap



- Bir Java kaynak dosyasında hiç sınıf olmayabilir ama bu durumda bu dosya derlendiğinde herhangi bir `class` dosyası oluşmaz

public Sınıf - II



- *Bir Java kaynak kodu dosyasında **public** olmayan pek çok sınıf bulunabilir dendi*
- Başka bir amaç güdülmüyorsa yönetim kolaylığı için tercih edilen, **public** olsun olmasın, her sınıfın kendi dosyasında tanımlanmasıdır
- Bir Java kaynak kodu dosyasında hiç **public** sınıf yoksa ya da sadece isimsiz/örtük bir sınıf varsa kaynak dosyasına herhangi bir isim verilebilir
- Ama bu da yönetim için zorluk çıkarır, dolayısıyla non-public sınıfları da kendi isimlerine sahip kaynak kod dosyalarına koymak kolaylık sağlar

Selam.java & SelamTest.java



```
public class Selam{
    String world = "millet";
    public String selamSoyle(String kime) {
        String cumle;
        if (kime != "")
            cumle = "Selam " + kime + " :)";
        else
            cumle = "Selam " + world + " :)";
        return cumle;
    }
}
```

```
public class SelamTest{
    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSoyle("Ali");
        System.out.println(cevap);
    }
}
```

- ch04 . selam dizinindeki Selam.java ve SelamTest.java isimli kaynak kod dosyalarında tanımlı sınıflar

Tek Dosyada Çok Sınıf Tanımı



- `Selam` ve `SelamTest` sınıfları, tek bir dosyada, örneğin `SelamTest.java` dosyasında tanımlanabilir
- Bu durumda `SelamTest` sınıfı `public` olarak bırakılıp, `Selam` sınıfının başındaki `public` anahtar kelimesi kaldırılmalıdır
- Bu durumda `Selam` sınıfı *non-public* olmuştur
- Bu iki sınıf, ancak `SelamTest.java` isimli dosyaya kaydedilirse hata almadan derlenebilir çünkü tek `public` sınıfın ismi `SelamTest`'tir

SelamTest.java



```
public class SelamTest{
    public static void main(String[] args){
        Selam nesne = new Selam();
        String cevap = nesne.selamSoyle("Ali");
        System.out.println(cevap);
    }
}

class Selam{
    String world = "millet";

    public String selamSoyle(String kime) {
        String cumle;
        if (kime != "")
            cumle = "Selam " + kime + " :)";
        else
            cumle = "Selam " + world + " :)";
        return cumle;
    }
}
```

- SelamTest.java isimli dosyada tanımlı, public olan SelamTest ve public olmayan Selam sınıfları

HelloWorld.java



```
import ch04.compact.Selam;

Random random = new Random();

void main() {
    System.out.println("Hello World!");
    saySalam();
}

void saySalam(){
    Selam s = new Selam();
    int limit = random.nextInt(10);
    for(int i = 0; i < limit; i++)
        System.out.println(s.selamSoyle("0ya"));
}
```

Dosya ismi: HelloWorld.java

- Java 25 ile gelen özellikte, eğer isimsiz/örtük (unnamed/implicit) bir sınıf tanımlanıyorsa bu durumda kaynak kod dosyasına istenilen isim verilebilir
- `ch04.compact` dizinindeki `HelloWorld.java` isimli dosyada tanımlı isimsiz/örtük bir sınıf vardır

Selam ve SelamTest



- `Selam` dizinindeki `Selam.java` ve `SelamTest.java` kaynak kod dosyalarındaki sınıfları tek bir dosyada, `SelamTest.java` dosyasında birleştirelim
- Bu durumda `public` sınıfın ismi de `SelamTest` olmalıdır
- Diğer sınıf, `Selam`, `public` olamaz, tanımında `public` kullanılmamalıdır

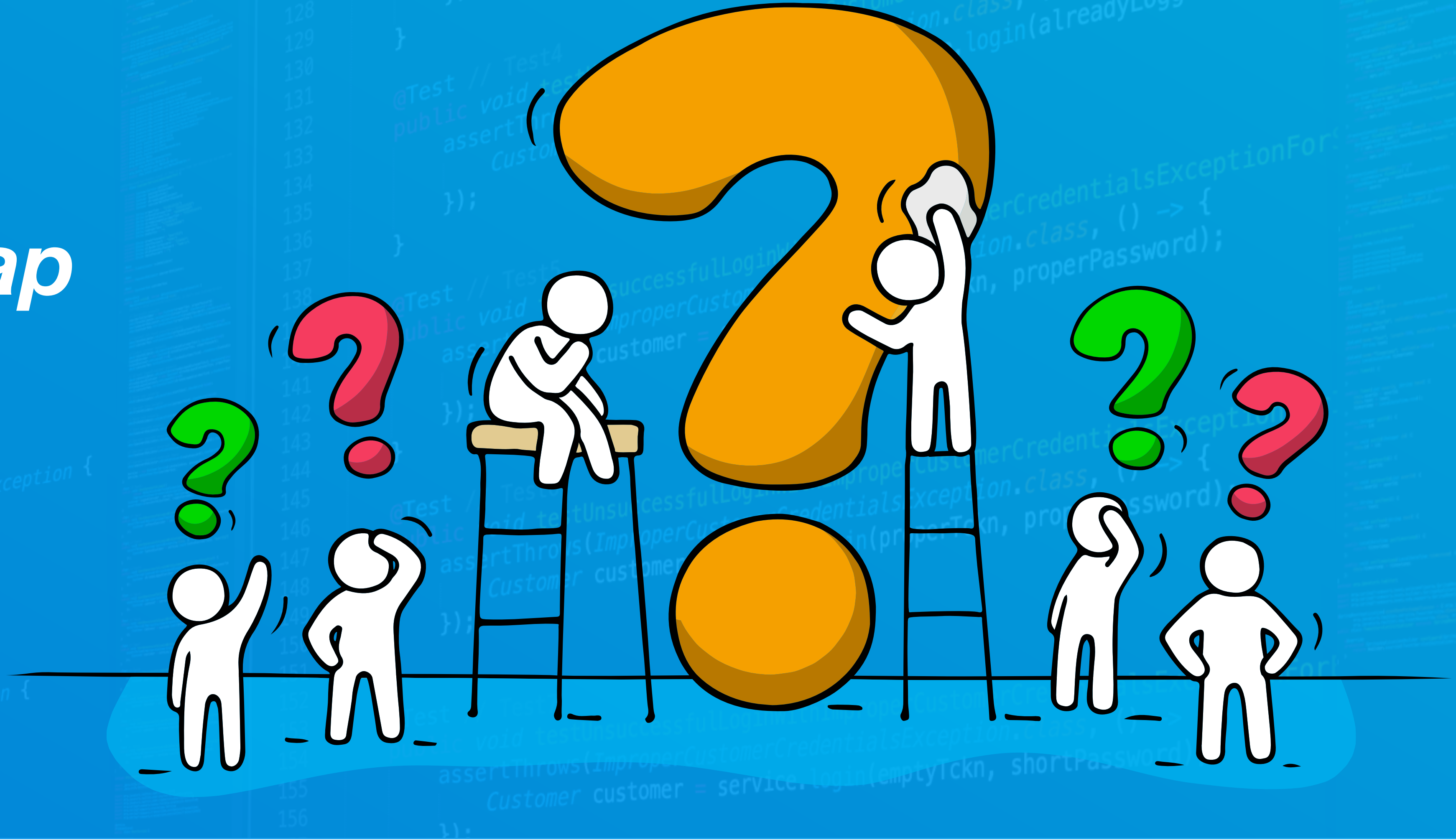
Selam ve SelamTest



- Komut satırında bu iki sınıf bir komutla derlenir: `javac SelamTest.java`
- Derlemeden önce kaynak dosyasında varsa `package` satırını kaldırın
- Derlemeden sonra iki tane `class` dosyasının, `Selam.class` ve `SelamTest.class` oluştuğunu gözlemleyin
- Ve çalıştırılır: `java SelamTest`

```
akin@M3-Max:~/desktop/Code/ch04/selam
/Users/akin/desktop/Code/ch04/selam >ls
SelamTest.java
/Users/akin/desktop/Code/ch04/selam >javac SelamTest.java
/Users/akin/desktop/Code/ch04/selam >ls
Selam.class      SelamTest.class SelamTest.java
/Users/akin/desktop/Code/ch04/selam >java SelamTest
Selam Ali :)
/Users/akin/desktop/Code/ch04/selam >
```

Soru ve Cevap Zamanı!





Derleme ve Çalıştırma

Çalıştırma Alternatif Yolları



- Java kaynak kodlarını çalıştırmanın iki yöntemi olduğunu biliyoruz:
 - Klasik derleme ve çalıştırma döngüsü: `javac` ile kaynak kod derlenir ve `class` dosyası oluşur, sonrasında `class` dosyası `java` ile çalıştırılır
 - Kaynak kodu çalıştırma: `java` ile kaynak kod doğrudan çalıştırılır
- İlk yöntem Java'nın ilk sürümünden beri vardır
- İkinci yöntem ise Java 11 ile dile katılmıştır ve ismine **source-file mode** denmiştir: <https://openjdk.org/jeps/330>

Derleme ve Çalıştırma - I



- Java kaynak dosyasının ismi ne olursa olsun derlendiğinde içinde tanımlı her sınıf için ayrı bir `class` dosyası oluşur
- Oluşan `class` dosyalarının isimleri, içerdikleri sınıfların isimleriyle aynıdır
- İsimsiz/örtük sınıflarda ise oluşan `class` dosyasının ismi, kaynak kod dosyasının ismiyle aynı olur
- Source-file mode çalıştırmada ise derlenmiş bytecodelar sadece bellekte oluşur, ayrı bir dosyaya kaydedilmez

Derleme ve Çalıştırma - II



- Çalıştırmak için `java` komutuna `main` metodun içinde olduğu sınıfın ismi geçilir ki bu isim aynı zamanda `class` dosyasının da ismidir
- Sınıf `public` olduğunda, o sınıfın ismi aynı zamanda kaynak dosyasının da ismi olur
- Source-file mode çalıştırmada da `java` komutuna `main` metodun içinde olduğu dosyanın ismi geçilir

MySelamTest



- Şimdi önce `SelamTest.java` sınıfında `SelamTest`'i tanımlarken kullanılan `public` kelimesini ve varsa `package` satırını kaldırın sonra da bu dosyayı apayrı bir isimle örneğin `MySelamTest.java` olarak kaydedin
- Şimdi derleyin: `javac MySelamTest.java`
- Hangi `class` dosyaları oluştu?
- Ve çalıştırın: `java SelamTest`

```
akin@M3-Max:~/desktop/Code/ch04/selam
~/Users/akin/desktop/Code/ch04/selam >ls
MySelamTest.java
~/Users/akin/desktop/Code/ch04/selam >javac MySelamTest.java
~/Users/akin/desktop/Code/ch04/selam >ls
MySelamTest.java      SelamTest.class
Selam.class
~/Users/akin/desktop/Code/ch04/selam >java SelamTest
Selam Zeynep :)
~/Users/akin/desktop/Code/ch04/selam >
```



- `MySelamTest.java` dosyasındaki sınıflardan birisini `public` yapın ve bu şekilde derleyin
- Derleme sırasında aldığınız hatayı yorumlayın
- Hata, `public` olan sınıfın kendi ismine sahip bir dosyada tanımlanmış olması gerektiğini söyleyecektir



- Aynı işleri IDE'de de yapabilirsiniz

Source-File Mode - I



- Daha önce ***public*** sınıf, *kendi ismini taşıyan bir dosyada yer almak zorundadır* dedik
- Bu kural Java kaynak kodu, `java` komutuyla doğrudan çalıştırıldığında yani source-file mode'da işletilmez
- Çünkü Java kaynak kodunun `java` komutuyla doğrudan çalıştırılması bir kolaylıktır, temelde deneme amaçlı yazılan kodları değerlendirmek için hızlıca çalıştırmada kullanılır ve `java` komutu bu kuralı işletmez

Source-File Mode - II



- Hatta source-file mode çalıştırmada Java kaynak kodunun ismi `java` uzantısıyla bitmek zorunda değildir, `txt` gibi herhangi bir uzantıyla bitebileceği gibi hiçbir uzantıya sahip olmayabilir
- Unutmayın, `javac` ile derlemede Java kaynak kod dosyasının ismi `java` uzantısına sahip olmak zorundadır
- Ama bu durumda `java` komutuna geçilen dosyanın bir kaynak kod dosyası olduğu `--source N` ile belirtilmelidir
- Buradaki `N` ise kaynak kodun geçerli olduğu Java sürümüdür



- `compact.main` paketindeki `SimpleSelam.java` dosyasında `SimpleSelam` isimli `public` bir sınıf vardır
- Şimdi bu dosyanın ismini `X.java` olarak değiştirin, dolayısıyla dosyanın ismi ile içindeki `public` sınıfın ismi farklı olsun
- Ve source-file mode ile çalıştırın: `java X.java`
- Kod çalışıp çıktığı komut satırına basacaktır

SimpleSelam



- Ama aynı dosyayı `javac` ile derlemeye çalıştığınızda hata alırsınız:
- `javac X.java`

```
akin@M3-Max:~/desktop/Code/ch04/compact/main
~/Users/akin/desktop/Code/ch04/compact/main >ls
Hello.java      X.java
~/Users/akin/desktop/Code/ch04/compact/main >cat X.java
// package org.javaturk.ipj.ch04.compact.main;// Exactly the same
as Selam.java in ch01

public class SimpleSelam {
    void main() {
        System.out.println("Simple Selam :) ");
    }
}%
~/Users/akin/desktop/Code/ch04/compact/main >java X.java
Simple Selam :)
~/Users/akin/desktop/Code/ch04/compact/main >javac X.java
X.java:3: error: class SimpleSelam is public, should be declared i
n a file named SimpleSelam.java
public class SimpleSelam {
    ^
1 error
~/Users/akin/desktop/Code/ch04/compact/main >
```

← Error



- Şimdi `x.java` dosyanın ismiddeki `.java`'yı silin, dosya ismi `x` olarak kalsın
- Ve çalıştırın: `java --source 25 x`
- Yine kod sorunsuzca çalışıp çıktıyı komut satırına basacaktır

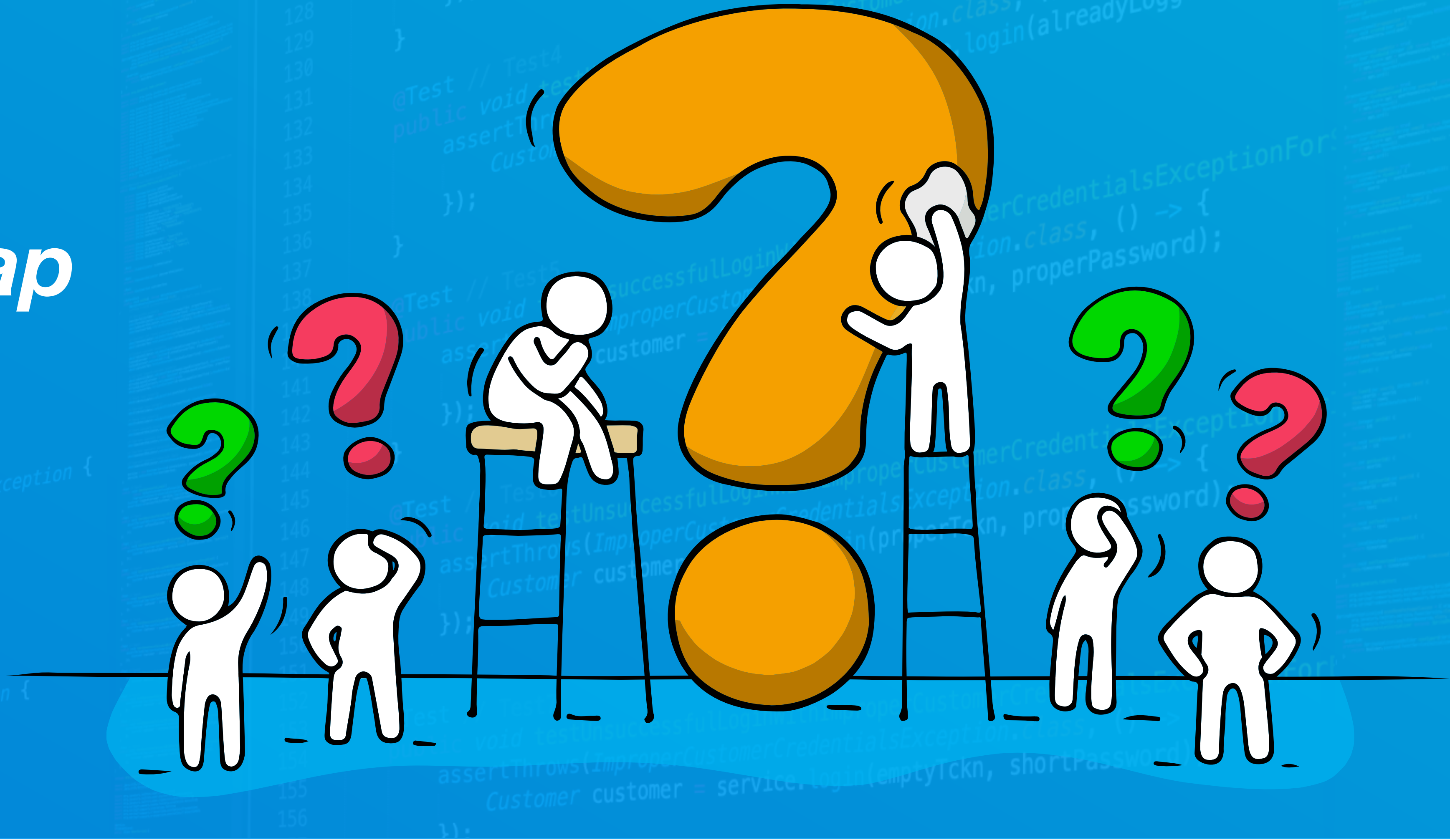


Ödev



- `hw.ch04.pi.MonteCarloPI` ve `MonteCarloPITest`
- Şu anda iki ayrı kaynak dosyasında tanımlanmış olan bu yapıyı tek bir dosyaya indirin ve derleyerek çalıştırın
- Sonra `MonteCarloPITest.java` dosyasının ismini `Test.txt` yapın ve source-file mode'da `java` ile doğrudan çalıştırın

Soru ve Cevap Zamanı!



Kısa Kaynak Dosyaları ve Nesne Main Metotları



Kısa Kaynak Dosyaları ve Nesne Main Metotları - I



- Java 21 ile birlikte ön izleme (preview) özelliği olarak **Unnamed Classes and Instance Main Methods (İsimsiz Sınıflar ve Nesne Main Metotları)** geldi
- <https://openjdk.org/jeps/445>
- Bu yenilik Java 25 ile birlikte **Compact Source Files and Instance Main Methods (Kısa Kaynak Dosyaları ve Nesne Main Metotları)** adı altında kalıcı hale geldi
- <https://openjdk.org/jeps/512>

Kısa Kaynak Dosyaları ve Nesne Main Metotları - II



- Bu özellikle sınıf olmadan sadece `main` metot kullanarak geçerli Java kodu oluşturmak mümkün hale geldi
- Malum aslında `package` ve `import` cümleleri dışında her şey bir sınıf içinde olmalıdır
- Bu özellikle, Java'yı öğrenmenin kolaylaştırılması ve giriş seviyesindeki karmaşıklığın azaltılması amaçlanmıştır
- Şimdi bu iki özelliği ele alalım



Kısa Kaynak Dosyaları

Kısa Kaynak Dosyaları - I



- Detaylarını az önce öğrendiğimiz gibi, tüm Java kodları sınıflar içinde olur, sınıflar paketler ve paketler de modüller içinde yer alır
- Paketler ve modüller ileriki konulardandır
- Fakat, öğrenme kolaylığı sağlamak için, küçük programlarda bunlara olan ihtiyaç ortadan kaldırılmıştır
- Dolayısıyla Python vb. dillerde olduğu gibi artık Java'da sınıf oluşturmadan bir program yazılabilir

Kısa Kaynak Dosyaları - II



- Java'da sınıf oluşturmada program yazabilmek için bir Java kaynak dosyasında geçerli bir `main` metot olmalıdır
- Geçerli `main` metodun özelliklerini birazdan ele alacağız
- Geçerli bir `main` metodun bulunmadığı durumda derleyici hata verir
- Dolayısıyla en kısa Java kodu ancak basit bir `main` metot ile yazılabilir

```
void main() {  
    IO.println("Hello");  
}
```

PrintEven



- `compact.PrintEven`
- `main` metodun ismini değiştirip sonucu gözlemleyin

Kısa Kaynak Dosyaları - II



- Java'da sınıf oluşturmada kod yazıldığında kaynak kodda bulunmasa bile derleyici arka planda, `class` dosyasında görünecek şekilde, bir sınıf yaratır ki bu sınıfa *implicitly declared top-level class* (örtük olarak tanıtılmış üst düzey sınıf) denir
- Bu şekilde tanımlanmış kaynak koda da *compact source file* (kısa kaynak dosyası) adı verilir
- Nitekim bu yenilikler **Compact Source Files and Instance Main Methods** (Kısa Kaynak Dosyaları ve Nesne Main Metotları) ismiyle dile katılmıştır

Kısa Kaynak Dosyaları - IV



- Örtük sınıfların (implicit classes) diğer bazı özellikleri şunlardır:
 - Örtük sınıflar pakete sahip olamaz
 - Örtük sınıflara başka bir sınıftan ulaşılamaz çünkü isimleri yoktur
 - Ama örtük sınıflar başka sınıfları `import` edip kullanabilir
 - Örtük sınıfların nesnesi oluşturulamaz çünkü isimleri yoktur

Kısa Kaynak Dosyaları - V



- Örtük sınıflar **main** metot dışında başka metotlarına sahip olabilir
- Örtük sınıflar değişkenlere sahip olabilir
- Örtük sınıfların değişken ve metotları nesne ya da sınıf için olabilir
- Bu ayrıma daha sonraki bölümlerde girilecektir

Kısa Kaynak Dosyaları - VI



- *Örtük sınıflara başka bir sınıftan ulaşılamaz çünkü isimleri yoktur* dendi, ama örtük sınıf başka sınıflara ulaşabilir gerekirse `import` edebilir
- Ayrıca *örtük sınıfta geçerli bir `main` metot olmalıdır* dendi
- Buradan örtük sınıfları ancak şu iki şekilde kullanabileceğimiz anlaşılır:
 - Ya tek dosyalık basit denemeler yapmak için, daha çok öğrenme amaçlı
 - ya da bir çok sınıftan oluşan bir yapıyı çalıştırmak için, bu çok daha genel amaçlı bir kullanımdır

Kısa Kaynak Dosyaları - VII



- Dolayısıyla örtük sınıfın varlık sebebi her halükarda bir `main` metoda sahip olmasıdır!
- Bu şekilde yazılmış kısa kaynak kod dosyaları `java` ile doğrudan çalıştırılabilir ya da `javac` ile derlenip `class` dosyaları oluşturulup sonrasında `java` ile çalıştırılabilir
- Kısa kaynak kod dosyası olarak yazılmış bir kod parçası `javac` ile derlendiğinde oluşan `class` dosyası ismini, kaynak dosyasından alır

Greeting.java — `javac` —> Greeting.class

Greeting



- IDE üzerinde `compact` paketindeki `Greeting.java` dosyasını çalıştırın

Greeting



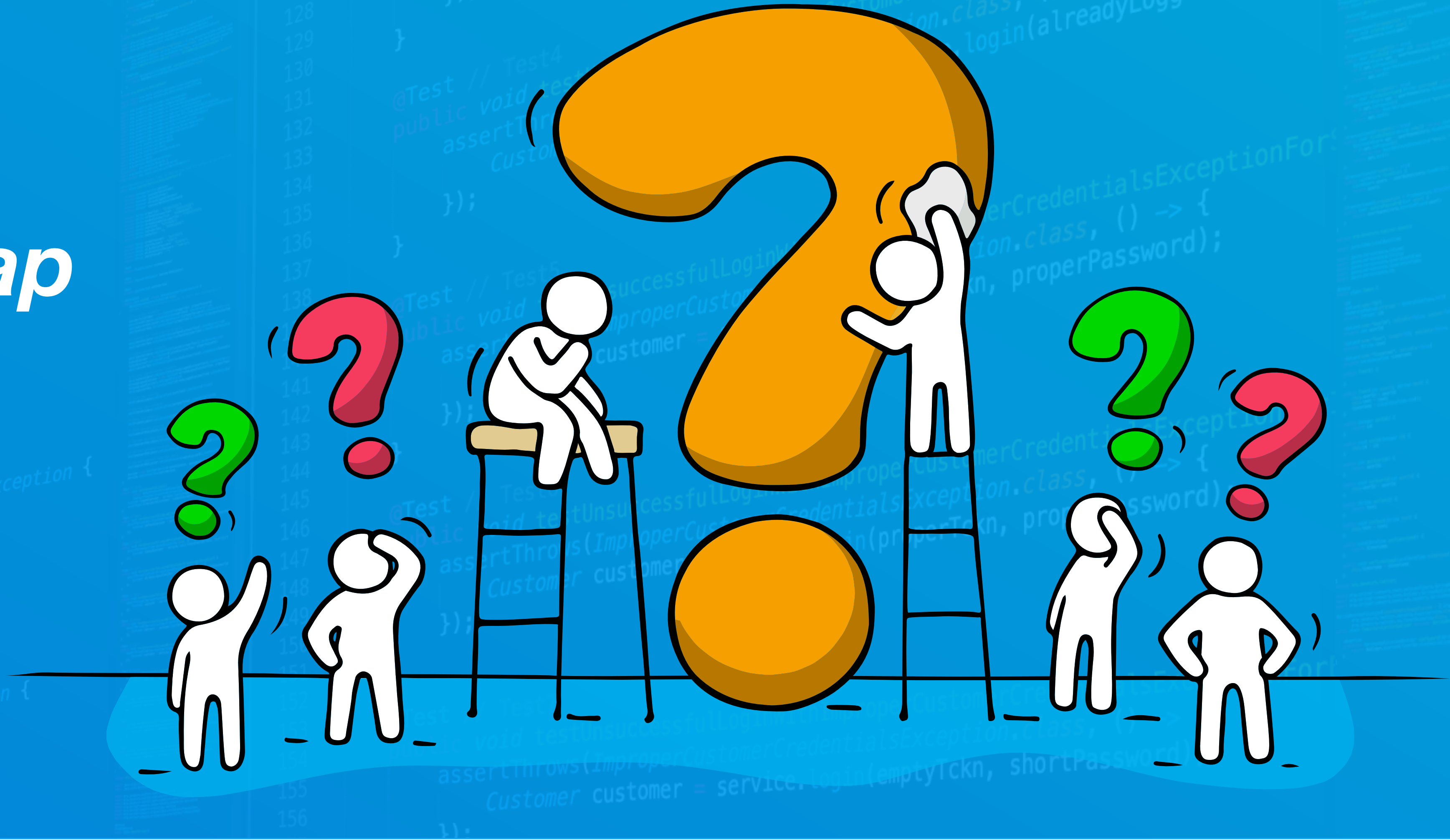
- Projenizdeki `compact` dizinini, masa üstündeki `Code/ch04` dizinine taşıyın
- Komut satırında `Greeting.java` dosyasını `java` ile doğrudan çalıştırın
- Bu amaçla `Selam.java` ve `Hello.java` dosyalarındaki `package` satırlarıyla `Greeting.java` dosyasındaki bu sınıfları `import` eden iki satırı kaldırın

Greeting



- `Greeting.java` dosyasını `javac` ile derleyip `java` ile çalıştırın
- İsimsiz sınıfa sahip olmasına rağmen `Greeting.java` dosyasından `Greeting.class` dosyasının üretildiğini gözlemleyin
- Ayrıca `Selam.class` ve `Hello.class` dosyalarının da üretildiğini gözlemleyin

Soru ve Cevap Zamanı!



main Metot



main Metot - I



- Java'da çalışma zamanı (run-time) daima `main` metot ile başlar
- `main` metot tüm kodun çalışmaya başladığı yerdir
- Java 25 öncesinde `main` metot daima aşağıdaki gibi yazılmalıdır
- Geçilen `String` tipindeki dizinin (array) yazımına dikkat edin

```
public static void main(String[] args)
```

```
public static void main(String [] args)
```

```
public static void main(String []args)
```

```
public static void main(String args[])
```

```
public static void main(String ... args)
```

main Metot - II



- Programı çalıştırmak için `java` komutuna, `main` metodunun içinde olduğu sınıfın ismi geçilir
- **Unutmayın, `javac` komutu kaynak dosyasının tam ismini alırken `java` sadece `main` metoda sahip sınıfın ismini alır**
- Ayrıca, eğer bir `java` uzantılı Java kaynak dosyasında birden fazla sınıf varsa, tabi ki sadece bir tanesi `public` olabilir ama her sınıf ayrı ayrı `main` metoda sahip olabilir

SelamTest.java & Selam.java



```
public class Selam{
    String world = "millet";

    public String selamSoyle(String kime) {
        String cumle;
        if (kime != "")
            cumle = "Selam " + kime + " :)";
        else
            cumle = "Selam " + world + " :)";
        return cumle;
    }
}
```

```
public class SelamTest{
    public static void main(String[] args) {
        Selam reference = new Selam();

        String cevap = reference.selamSoyle("Ali");
        System.out.println(cevap);

        cevap = reference.selamSoyle("");
        System.out.println(cevap);
    }
}
```

- `ch04.selam` dizinindeki `SelamTest` sınıfının `main` metodunda `Selam` sınıfının bir nesnesi oluşturulmakta, **reference** isimli bir referansa atanmakta ve üzerinde `selamSoyle()` metodu iki defa, iki farklı argümanla çağrılmakta ve dönen cevaplar yazdırılmaktadır



- Şimdi Java 25 ve öncesindeki `main` metodun yazılışını deneyerek görelim
- Bu amaçla `compact.main` paketindeki `SimpleSelam.java` dosyasını komut satırında çalışacak şekilde, paket satırını silerek hazır hale getirin
- Sonra Java 25 kullanarak komut satırında `SimpleSelam.java` dosyasını aşağıdaki şekildeki `main` metotlarla `java` komutuyla çalıştırın:



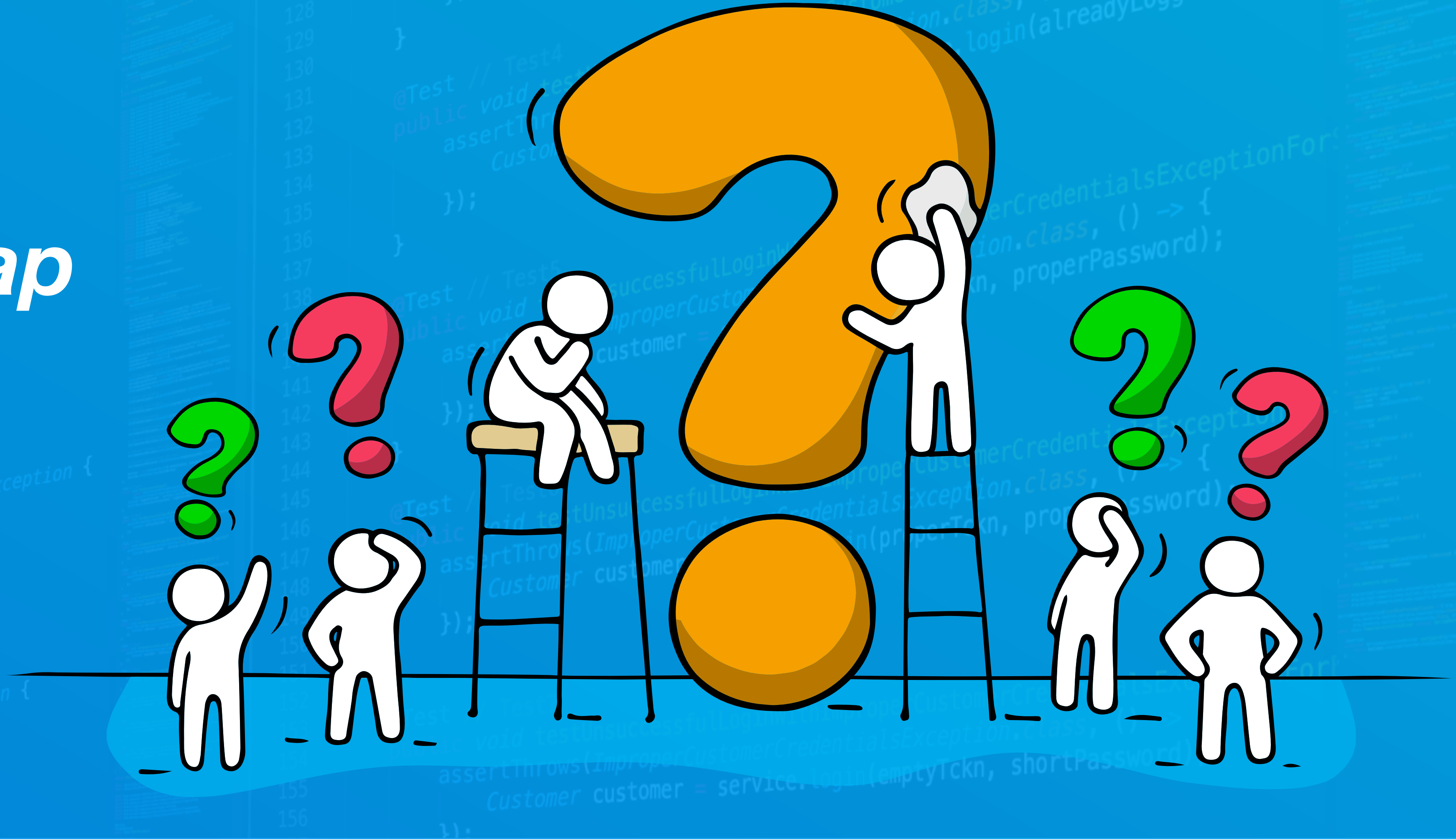
- `public static void main(String[] args)`
- `static void main(String[] args)`
- `public void main(String[] args)`
- `void main(String[] args)`
- `public static void main()`
- `static void main()`
- `void main()`

SelamTest.java & Selam.java



- Daha sonra aynı şeyi Java 25 öncesi bir sürümle mesela Java 21 ile yapın
- Gözleminiz nedir? Ne zaman hangi hataları aldınız? Sebebini açıklayın
- `SimpleSelam.java`'yı farklı bir `main` metotla örneğin `public` kelimesini düşürerek derleyip çalıştırın

Soru ve Cevap Zamanı!





Nesne main Metotları

Nesne main Metotları - I



- Java 25 ile gelen **Nesne Main Metotları (Instance Main Methods)** özelliğiyle `main` metodu nesne metodu olarak tanımlanabilir: <https://openjdk.org/jeps/512>
- Dolayısıyla `static` kullanılmaz (detaylarını ileride inceleyeceğiz)
- `public` anahtar kelimesi de zorunlu olmaktan çıkmıştır
- `String[]` argümanı kullanımı tamamen ihtiyaca bağlıdır (detaylarını ileride ele alacağız)

```
public static void main(String[] args)
```

Nesne main Metotları - II



- Dolayısıyla kısa kaynak dosyalarında bu iki `main` metottan birisi kullanılabilir

```
void main()
```

```
void main(String[] args)
```

- Bu şekilde tanımlanan `main` metot, nesnenin bir metodudur ve arka tarafta oluşturulan örtük sınıfın (implicit class) örtük bir şekilde yaratılan nesnesi (implicit instance) üzerinde çağrılır
- Bu özellik de Java'ya girişi kolaylaştırmak ve Java'nın eleştirilen merasimini (ceremony) azaltmak için geliştirilmiştir

Nesne main Metotları - III



- Eğer `main` metot tanımında `static` kelimesi kullanılırsa bu durumda örtük sınıfın bir nesnesi oluşturulmadan `main` metot doğrudan çağrılır

```
static void main()
```

- `static` kelimesi de ileride detaylıca ele alınacaktır
- Benzer şekilde `String[]` argümanı yine ihtiyaca göre kullanılabilir
- Dolayısıyla `public` olmadan aşağıdaki `main` metot ta kullanılabilir

```
static void main(String[] args)
```

Nesne main Metotları - IV



- Dolayısıyla bir kısa kaynak kod dosyasında aşağıdaki sekiz `main` metottan birisi yoksa hata alınır:

Nesne main Metotları

En kısa



```
void main()
```

Argümanlı
en kısa



```
void main(String[] args)
```

Gereksiz,
çünkü
`public`
zorunlu değil



```
public void main()
```



```
public void main(String[] args)
```

Sınıf main Metotları

```
static void main()
```



```
public static void main()
```



```
static void main(String[] args)
```



```
public static void main(String[] args)
```



Gereksiz,
çünkü
nesne main
metotları
daha basit

Artık kim
yazar bunu?

Nesne main Metotları - V



- Bir kaynak kod dosyasında imzası aynı iki farklı `main` metot olamaz, birisi `main()` ise diğeri `main(String[])` olmalıdır
- Bu durum metotların overload edilmesiyle ilgilidir ve ileride ele alınacaktır
- Bir kaynak kod dosyasında, `main()` ve `main(String[])` şeklinde, imzası farklı iki `main` metot varsa, `String[]` argüman alan çağrılır
- Ayrıca `main` metot `private` olamaz, `protected` olabilir
- Bu iki erişim kontrol yapısı da ileride ele alınacaktır

Nesne main Metotları - VI



- Şu ana kadar `main` metot için belirttiğimiz yenilikleri kısa kaynak kod dosyasında tanımlanması için söyledik
- Aslında tüm bu farklı `main` metotlarından herhangi birini, bir sınıfın içinde de kullanabilirsiniz
- Peki ama niye koyalım ki sınıf olmadan kod yazmak varken?
- Dolayısıyla kısa denemelerimizde ya da pek çok kaynak kod dosyası ve pek çok sınıftan oluşan sistemlerimizi çalıştırmak için bir kaynak kod dosyasına basit bir `main` metot koymak artık yeterlidir



- Sonuç olarak Java 25 ve daha yeni sürümlerle çalışırken **main** metodunu en basit halleriyle ve sınıf tanımlamadan yazalım:

```
void main()
```

```
void main(String[] args)
```

- Java 25 öncesi projelerde zaten sınıf da tanımlamak zorundayız, **main** metodu da bilindik haliyle yazmak zorundayız:

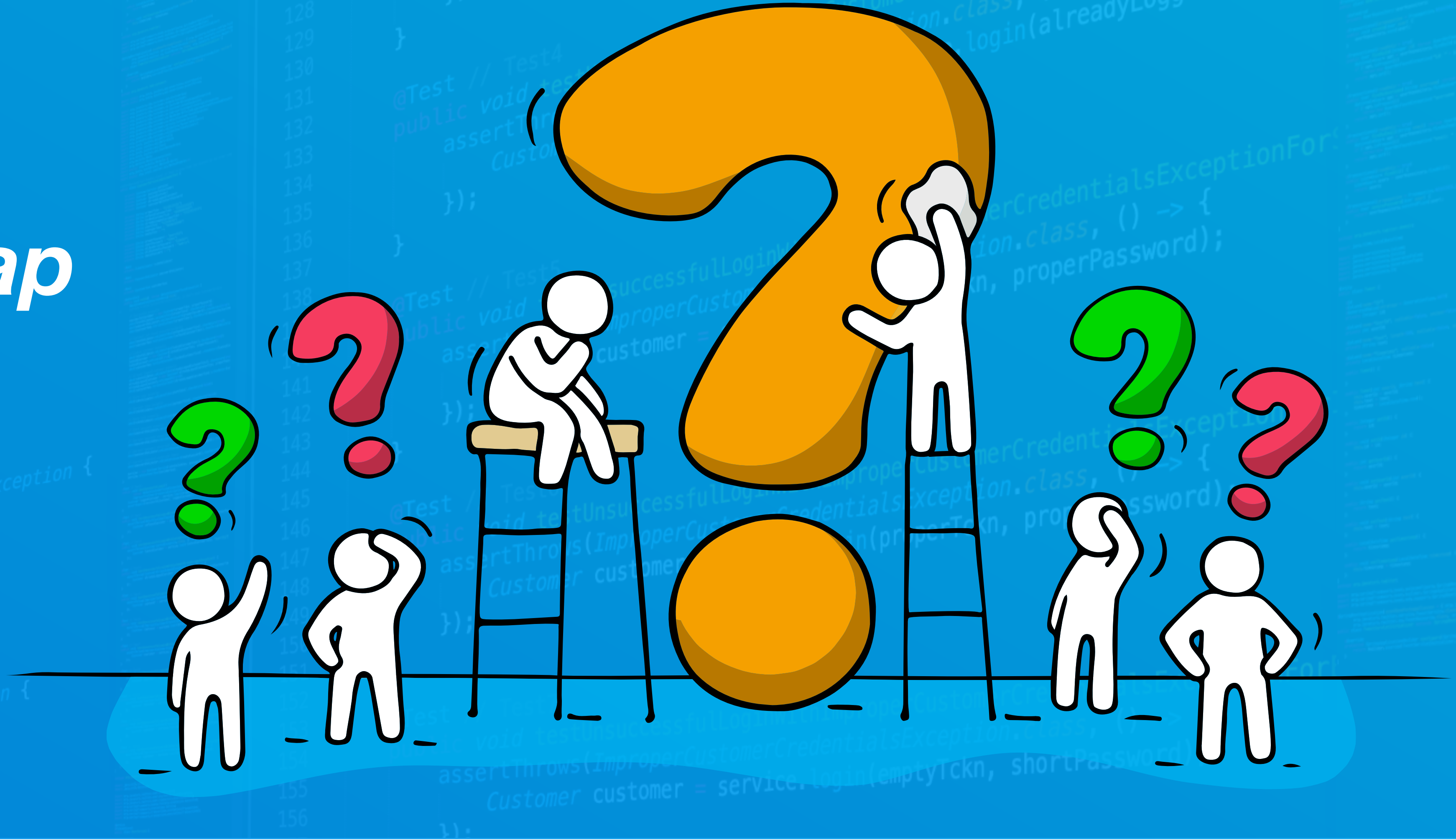
```
public static void main(String[] args)
```

Hello



- `compact.main.Hello`
- `main` metodun farklı kullanımları ile yazıp çalıştırın
- Birden fazla farklı `main` metot ile olan davranışı gözlemleyin
- Sınıf tanımını kaldırıp sadece `main` metotların olduğu halleri de çalıştırın
 - Bu durumda `package` cümlesinin de kaldırmanız gerekir
- Bunları hem IDE hem komut satırında yapın

Soru ve Cevap Zamanı!



Ödevler



Ödev - I



- Aşağıdaki her satırın ifade mi cümle mi yoksa hiçbirisi mi olduğuna karar verin

- `x + y * 2`

- `int age = 25`

- `age = 25`

- `int age = 25;`

- `(temperature > 100)`

- `return false;`

- `score >= 90 ? "A" : "B"`

- `System.out.println("Hi");`

- `{`

- `method(x, y)`

- `break;`

Ödev - II



- `hw.ch04` paketindeki `Rectangle.java` ve `RectangleTest.java` kaynak kodlarını inceleyin
- `RectangleTest.java` dosyasındaki `main` metotta başka bir `Rectangle` nesnesi oluşturup bilgileriyle, alanını ve çevresini hesaplatıp bastırın
- `RectangleTest.java`'yı çalıştırın

Ödev - III



- `hw.ch04` paketindeki `Rectangle.java` ve `RectangleTest.java` örneğine benzer bir şekilde `Circle` ve `Circle Test` sınıflarını yazıp çalıştırın

Ödev - IV



- `hw.ch04.name` paketindeki `NameGatherer.java` dosyasını inceleyin ve hem komut satırında hem de IDE'de çalıştırın

Ödev - V



- `hw.ch04` paketindeki `MainMethods.java` sınıfındaki `main` metotları inceleyin ve sırayla çalıştırın
- Hangilerinin geçerli hangilerinin geçersiz olduğunu belirleyin

Ödev - VI



- Al'ya, Java 25 öncesi ve sonrası için yazılıp çalıştırılabilecek en kısa Java kodunu sorun.

Bundan Sonra?



Bundan Sonra?



- Bundan sonra Java'nın tip sistemini inceleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

